

Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Várnagy Erik Zoltán

2026

Szegedi Tudományegyetem
Informatikai Intézet

SocialBoost-AI applikáció

Szakdolgozat

Készítette:

Várnagy Erik Zoltán

üzemtechnológus-informatikus
BProf szakos hallgató

Témavezető:

Dr. Bilicki Vilmos

egyetemi docens

Szeged

2026

Feladatkiírás

A szakdolgozat célja egy olyan alkalmazás, amely magyar kis és középvállalkozók internetes jelenlétét hivatott növelni különböző segítő funkcióival. Az alkalmazás egyaránt működik webes és mobilos böngészőkön, nem szükséges külön telepíteni. Így bárhol, bármely eszközről elérhető a felhasználó számára.

Az applikáció lehetővé teszi a felhasználók számára, hogy posztötleket generáljanak, konkrét posztokat készítsenek, és azokat egy könyvtárban elmentsék későbbre. A rendszer ezen felül támogatja a heti poszttervek előre generálását, így a felhasználó könnyen tervezheti közösségi média tevékenységét. A generált posztötlek és tartalmak felhasználóhoz kötve kerülnek mentésre, a mentett adatok pedig felhőalapú, nem-relációs (NoSQL) adatbázisban tárolódnak, ami biztosítja az adatok gyors és skálázható kezelését és titkosítását.

A Profil menüpontban a felhasználó megadhatja üzletének és célközönségének tulajdonságait, amelyek alapján a mesterséges intelligencia alapú generátor testreszabott, az adott üzletághoz és célcsoporthoz illeszkedő posztötleket hoz létre. A profiladatok egyszeri megadása elegendő, de szükség esetén később is módosíthatóak.

A felhasználói fiók létrehozása vagy belépés kötelező az alkalmazáshoz, mivel a generált tartalmak így lesznek optimálisak és személyre szabottak, ez az egész applikáció lényege.

Tartalmi összefoglaló

A téma megnevezése:

Webes és mobil böngészőben működő, mesterséges intelligencián alapuló alkalmazás kis és középvállalkozók számára.

A megadott feladat megfogalmazása:

A feladat egy olyan alkalmazás elkészítése, mely segíti a felhasználót a közösségi média tartalomtervezésében. Az applikáció lehetővé teszi konkrét posztok és posztötletek generálását, valamint azok könyvtárban való mentését és törlését későbbi felhasználási célból. Továbbá a rendszer heti terv generálására is képes a profiladatok alapján.

A megoldási mód:

A fejlesztés első lépéseként a fő funkciókat és felhasználói folyamatokat Wireframe-ek segítségével terveztem meg. Ezt követően MVP (Minimum Viable Product) szelet került kialakításra, amely a rendszer alapvető funkcióit – posztötlet generálás, bejelentkezés, profilmezők kitöltése tartalmazza mockolt adatokkal –. Az MVP jelentősen hozzájárult a későbbi fejlesztési problémák elkerüléséhez, mivel egy jól definiált alapstruktúrát biztosított, amelyre skálázható módon lehetett építeni.

Alkalmazott eszközök és módszerek:

Az alkalmazás frontendje Angular keretrendszerben készült. A backend oldali funkcionalitást egy saját fejlesztésű, Node.js és Express alapú REST API biztosítja, amely mesterséges intelligencián alapuló tartalomgenerálást, az üzleti logika kezelését, valamint API kulcsok biztonságos kezelését valósítja meg. Az adatok tárolása és felhasználók autentikációja a Google Firebase szolgáltatásain keresztül történik, Cloud Firestore adatbázis felhasználásával.

Elért eredmények:

Eredményként egy letisztult, modern megjelenésű webes és mobil böngészőben egyaránt használható alkalmazás készült el, mely lehetőséget ad a felhasználók számára posztötletek generálására, azok rendszerezésére, mentésére és tervezésére. Az alkalmazás teljes mértékben személyre szabott, könnyen értelmezhető és kezelhető.

Kulcsszavak:

Angular, Node.js, Express.js, Firebase, Webalkalmazás, API

Tartalomjegyzék

Feladatkiírás	2
Tartalmi összefoglaló	3
Motiváció	7
1. FELHASZNÁLT TECHNOLOGIÁK	8
1.1. Angular.....	8
1.1.1. TypeScript	9
1.2. Firebase	9
1.2.1. Cloud Firestore	10
1.3. Express.js.....	10
1.4. Node.js.....	10
1.5. OpenAI API.....	11
1.6. ZOD.....	11
2. PIACKUTATÁS	12
2.1. Predis.ai	13
2.2. Canva.....	13
2.3. Ocoya	13
2.4. Simplified	14
3. FUNKCIONÁLIS SPECIFIKÁCIÓ	15
3.1. Bevezetés.....	15
3.2. Autentikáció	17
3.2.1. Regisztráció	17
3.2.2. Autentikált bejelentkezés	19
3.2.3. Jogosultságkezelés	20
3.2.4. Kijelentkezés	21
3.3. Az alkalmazás fő funkciói.....	21
3.3.1. Kezdőoldal	21
3.3.2. Üzletprofil oldal	22
3.3.3. Rövid posztötlet és komplett posztszöveg generáló oldal.....	24
3.3.4. Heti tervező oldal	24
3.3.5. Képgenerálás oldal	25
3.3.6. Könyvtár oldal.....	26
4. BELSŐ FELÉPÍTÉS	28
4.1. Komponensek.....	28

4.2 Szolgáltatások (Services)	29
4.3 Adatmodellek és típusok	30
4.4 Routing és jogosultságkezelés.....	31
4.5 Kliens-szerver integráció.....	33
4.6 Firestore-ban tárolt adatok	33
4.7 Belső működési összegzés	34
5 SAJÁT API BEMUTATÁSA	35
5.1. Az API szerepe a rendszerben.....	35
5.2. Technológiai alapok	35
5.3. API végpontok áttekintése	36
5.4. Bemeneti validáció.....	37
5.5. Végpont-specifikus működés	38
5.6. Fallback stratégia és rendelkezésre állás	39
5.7. Promptolási stratégia az API-ban	39
5.8. Környezeti konfiguráció.....	39
5.9. Összegzés	40
6. A RENDSZER MAGASSZINTŰ FOLYAMATAI	41
7. Mesterséges intelligencia használata a fejlesztés során	43
8. Tesztelés	45
Irodalomjegyzék	47
Nyilatkozat.....	48
Elektronikus melléklet	49

Motiváció

A szakdolgozatom témájának kiválasztásakor egy olyan problémát szerettem volna keresni és megoldást nyújtani rá, ami valóban jelen van gyorsan rohanó világunkban és mindennapi szinten is releváns. Napjainkban az internet és a közösségi média kiemelt szerepet tölt be az emberek nagy részének mindennapi életében, és a kis és középvállalkozások életében is. Sok vállalkozó nem használja a közösségi médiát, avagy nem ért a közösségi médiához, vagy ideje se kedve vele foglalkozni.

Motivációm az adta, hogy egy olyan alkalmazást hozzak létre, amely mesterséges intelligencia segítségével lerövidíti a közösségi média kezelésére fordított időt, ezáltal gyorsabban és hatékonyabban tudja a vállalkozás szinte bármilyen célját elérni a közösségi média profilk felfuttatásával. Fontos szempont volt még, hogy rengeteg marketinghez és közösségi médiához nem értő vállalkozó van, emiatt fontos volt, hogy az alkalmazás segítsen irányt mutatni, és konkrét posztolható tartalmakat is adni, és skálázhatóságot biztosítani a heti tervezővel.

Főleg Magyarországon, rengeteg olyan vállalkozó van, akinek erre szüksége van az előbb leírtak okán. Manapság egy vállalkozást szinte csak a folyamatos és aktív internetes jelenléttel lehet igazán sikeressé és népszerűvé tenni, viszont sokak ezt még mindig nem tudják, vagy ha tudnak is róla, nem kívánják ennek szentelni az idejüket, hogy konkrétan mi és hogyan kell a terjeszkedéshez, itt azonosítottam a piaci lehetőséget. A másik oka még az volt a választásomnak, hogy a virtuális piacon, és oldalak/alkalmazások közt szétnézve relatíve kevés konkurencia található.

Mindig is érdekelt mind a vállalkozások, mind a közösségi média, így számomra ez egy kifejezetten érdekes téma.

1. FELHASZNÁLT TECHNOLOGIÁK

A következő részben az alkalmazás fejlesztése során felhasznált technológiák bemutatása következik.

1.1. Angular

Az Angular egy TypeScript alapú, nyílt forráskódú frontend keretrendszer, melyet a Google fejleszt és tartja karban. Elsődleges célja nagyobb méretű, jól struktúrált és fejleszthető egyoldalas webalkalmazások fejlesztésének támogatása. Az alkalmazásom frontend része Angular keretrendszer segítségével került megvalósításra. Azért erre a technológiai döntésre jutottam mivel az Angular egyik főbb jellemzője, hogy komponens alapú, ezért a vizuális megjelenítést és a logikai részt nagyon jól elkülöníti.

Főbb előnyei még ennek az architektúrának, hogy a projekt külön kezelhető részekre szedhető szét, ezáltal a kód újrafelhasználható, skálázható és jól karbantartható, és hosszú távon bővíthető. Előnye még a keretrendszernek hogy beépített támogatást nyújt az oldalak közötti váltást útvonalkezeléssel, ami biztosítja az alkalmazáson belüli navigációt újratöltés nélkül, illetve HTTP kommunikációhoz is, amelyen keresztül a frontend oldal kapcsolatba lép a backend oldali REST API-val. Az Angularban is elérhető a dependency injection, mely lényege, hogy a komponensek nem közvetlenül hozzák létre a függőségeiket, hanem az Angular maga injektálja őket futásidőben. Ez a megközelítés szintén növeli a kód újrafelhasználhatóságának mértékét, tesztelhetőségét és karbantarthatóságát.

A zökkenőmentes működés és a jó felhasználói élmény érdekében az Angular az RxJS Observable alapú aszinkron adatkezelést támogatja. Ez lehetővé teszi, hogy az API-hívások, felhasználói események és egyéb időben változó adatfolyamok nem blokkoló módon legyenek kezelhetők. Ennek köszönhetően a felhasználói felület reszponzív marad, miközben a háttérben futó műveletek eredményei folyamatosan feldolgozhatók. Az egyik legnagyobb versenytársa a React, melyet a Meta fejleszt, ám a React egy komponensalapú JavaScript könyvtár, az Angular pedig komplett frontend keretrendszer. Emiatt az Angulart folyamatosan fejlesztik, rengeteg újdonság jelenik meg egy hónapon belül. Az Angular előnye még az Angular CLI, melyet alkalmazásom készítése közben is folyamatosan használtam. Megkönnyíti a hibakeresést, projekt létrehozását és konfigurálását.

1.1.1. TypeScript

A TypeScript egy nyílt forráskódú, JavaScriptből kifejlesztett és ráépülő programozási nyelv, melyet a Microsoft fejleszt. A TypeScript fő célja volt a JavaScript továbbfejlesztése statikus típusosság bevezetésével, amely lehetővé teszi a fejlesztési hibák korai felismerését már fordítási időben. A TypeScript fordítás során átalakul JavaScript kóddá, hogy a modern környezetekben és webböngészőkben használható legyen. Támogatja az objektumorientált programozás alapelveit, mint osztályok, interfészek, öröklődés, hozzáférési szinteket. Ezek használata hozzájárul a kód jobb strukturáltságához, hosszú távon fejleszthetőségéhez, olvashatóságához. Mivel az Angular keretrendszer erre a nyelvre épül, így ezzel lett megvalósítva az alkalmazás. Előnye volt a típusosságnak hogy egyszerűsítette a futásidejű hibák kiküszöbölését.

1.2. Firebase

A Firebase egy Google által fejlesztett felhőalapú platform, amely komplett backend infrastruktúrát biztosít webes és mobilalkalmazások számára. Célja, hogy leegyszerűsítse a fejlesztést kész szerveroldali funkcionálisok megvalósításához. A Firebase főbb funkciói többek között felhasználói autentikáció, adatbázis-kezelés, felhőalapú hosztolás. Szoros integrációt biztosít más Google szolgáltatásokkal, valamint támogatást nyújt frontend keretrendszerekhez mint például az Angular és React. Előnye még alkalmazásfejlesztés szempontjából az egyszerűen integrálható autentikációs szolgáltatás, és felhasználói fiókok egyszerű létrehozását és kezelését. Emellett nagyon jó a valós idejű adatkezelési képessége, emiatt jó választás kisebb és nagyobb alkalmazásokhoz.

1.2.1. Cloud Firestore

A Cloud Firestore a Firebase részeként elérhető nem relációs típusú dokumentumorientált adatbázis, amelyet kis- és nagy alkalmazásokhoz egyaránt érdemes használni. Az adatokat dokumentumokban és kollekciókban tárolja, ez rugalmas adatstruktúrát és egyszerű lekérdezhetőséget biztosít. Egyik fő előnye a valós idejű adatkezelés, amely lehetővé teszi az adatok folyamatos és automatikus szinkronizálását. A rendszer biztosítja az adatbiztonságot mind a tárolást adatátvitel során, valamint kötelezően konfigurálandó security rules-t alkalmaz, és szabályozható jogosultságkezelést. Automatikusan skálázódik terhelés függvényében, így alkalmas mindenféle méretű alkalmazáshoz.

1.3.Express.js

Az Express.js egy nyílt forráskódú, rugalmas webalkalmazás keretrendszer a Node.js platformhoz. Főbb jellemzői hogy minimalista és rugalmas, könnyű és gyors, nagy bővítményökoszisztéma. Lehetővé teszi HTTP kérések és válaszok kezelését, az útvonalak definiálását, valamint köztes rétegek használatát, és ezek segítségével a kérések gyorsan feldolgozhatók. Előnye a modularitás, amely segítségével a fejlesztők saját komponenseket hozhatnak létre, illetve különböző könyvtárakat és middleware-eket integrálhatnak, így gyorsan skálázható és jól karbantartható backend rendszerek hozhatók létre.

1.4.Node.js

A Node.js is egy nyílt forráskódú, platformfüggetlen JavaScript futtatókörnyezet, amely lehetővé teszi a JavaScript kód szerveroldali futtatását a böngészőn kívül. A Google által fejlesztett V8 JavaScript motorjára épül, és eseményvezérelt, nem blokkoló I/O modellt használ, ami rendkívül gyors és skálázható alkalmazások fejlesztését teszi lehetővé. A Stack Overflow szerint a Node.js az egyik leggyakrabban használt webes technológia. A JavaScript kód szerveren történő futtatásának képességét gyakran használják dinamikus weboldaltartalom létrehozására, mielőtt az oldal elküldésre kerülne a felhasználó böngészőjébe.

1.5. OpenAI API

Az OpenAI API egy olyan felhőalapú szolgáltatás, ami lehetővé teszi a fejlesztők számára, hogy az OpenAI által kifejlesztett, mesterséges intelligencia modelleket integráljanak a saját szoftvereikbe, alkalmazásaikba, és weboldalaikba. Maga a cég hozzáférést biztosít a GPT szövegalkotó széria mesterséges intelligencia modellekhez, a DALL-E képgenerálást és a Whisper beszédfelismerést lehetővé tevő modelleihez. A szolgáltatások RESTful API-n keresztül érhetőek el, így rugalmasan használhatóak bármely programozási nyelven. Ezekkel az API-kal rengeteg funkcióval bővíthetik a fejlesztők az alkalmazásaikat, saját hatalmas igényű hardveres infrastruktúra nélkül, hisz a számítási és egyéb feladatok az OpenAI szerverein futnak. A dolgozatban bemutatott alkalmazás backendje OpenAI hívásokat használ a posztok, posztötletek és heti terv generálására. A rendszerben jelenleg a gpt-4o-mini modell kerül alkalmazásra, a költséghatékonyság miatt.

1.6. ZOD

A Zod egy TypeScript-orientált séma-deklarációs és validációs könyvtár, amely elérhetővé teszi a fejlesztők számára hogy statikus típusokat és futásidejű ellenőrzéseket definiáljanak. Míg maga a TypeScript statikus ellenőrzést biztosít a fordítási időben, a külső forrásokból érkező adatok validálására is szükség van, itt jön a Zod hasznossága a képbe. Az én alkalmazásomban a ZOD a backend API-ba érkező kérések tartalmát ellenőrzi még azelőtt, hogy dolgozna velük az alkalmazás, ezzel megakadályozva azt, hogy hibás vagy hiányos adat menjen tovább az OpenAI felé.

2. PIACKUTATÁS

A szakdolgozatom elkészítése előtt szerettem volna tájékozódni arról, hogy a valós piacon milyen hasonló alkalmazások léteznek. Ezért piackutatást végeztem, mivel nem szerettem volna egy olyan alkalmazást készíteni, amelyet már több cég lefejlesztett, egyedi termék elkészítése volt a célom. Olyan webalkalmazásokat kerestem, amelyek funkcionálisan hasonlóak lehetnek az enyémhez. A vizsgálat során olyan webalkalmazásokat elemeztem, amelyek a közösségi média tartalomgyártás és -kezelés területén mesterséges intelligenciát (AI) alkalmaznak.

Szempont	Predis.ai	Canva	Ocoya	Simplified
Gazdasági modell	Előfizetéses	Freemium (Ingyenes alap)	Előfizetéses (éves díjjal olcsóbb)	Freemium (erősen korlátozott)
Ingyenes használat	Csak korlátozott próbaidő	Erős ingyenes csomag, de AI funkciók limitáltak	Nincs	Nagyon szűkös keret
Strukturált szöveg (Hook/CTA)	Nem (egybefüggő szöveg)	Nem (kontextus nélküli)	Nem (sablonos)	Nem (felületes)
Magyar nyelvi minőség	Alacsony	Közepes	Alacsony	Alacsony
Iparág-specifikus profil	Van	Nincs	Nincs	Nincs
Fő fókusz (UI)	Social media automatizáció	Általános grafikai design	Social marketing és ütemezés	Multifunkciós kreatív platform

2.1 ábra : Összefoglaló a hasonló alkalmazásokról

2.1. Predis.ai

A vizsgált megoldások közül a Predis.ai mutatja a legnagyobb funkcionális átfedést a saját fejlesztésemmel. A platform integrált módon képes szöveges bejegyzések, vizuális kreatívok (képek és videók), valamint releváns metaadatok generálására. Profil beállítási oldala is van. Ingyenes próbaideje van az alkalmazásnak, de nincs valódi élesen használható ingyenes csomagja. Tartalommarketing szempontjából jelentős hiányossága, hogy a generált szövegek strukturálatlanok, hiányzik a szakmailag indokolt hook (figyelemfelkeltés) és CTA (cselekvésre ösztönzés) szerinti szétbontás. Emellett a magyar nyelvi támogatása alacsony szintű, a kimeneti szövegek stílusa idegen a hazai piaci kontextustól.

2.2. Canva

Piacvezető design eszköz, hatalmas template könyvtár, ChatGPT-hez hasonló felületű kép és szöveg generáló oldal. Ingyenes alap csomag elérhető. Limitációi, hogy nem social-writing-first eszköz, az AI szöveg másodlagos funkció, nem strukturált, nincs külön poszt-

Hook/CTA rész. Nem tud semmit az adott felhasználó üzletágáról, minden generálás lényegében kontextus nélküli. Sok más funkció is van benne, az alkalmazás fő fókusza nem ezen a részen alapul.

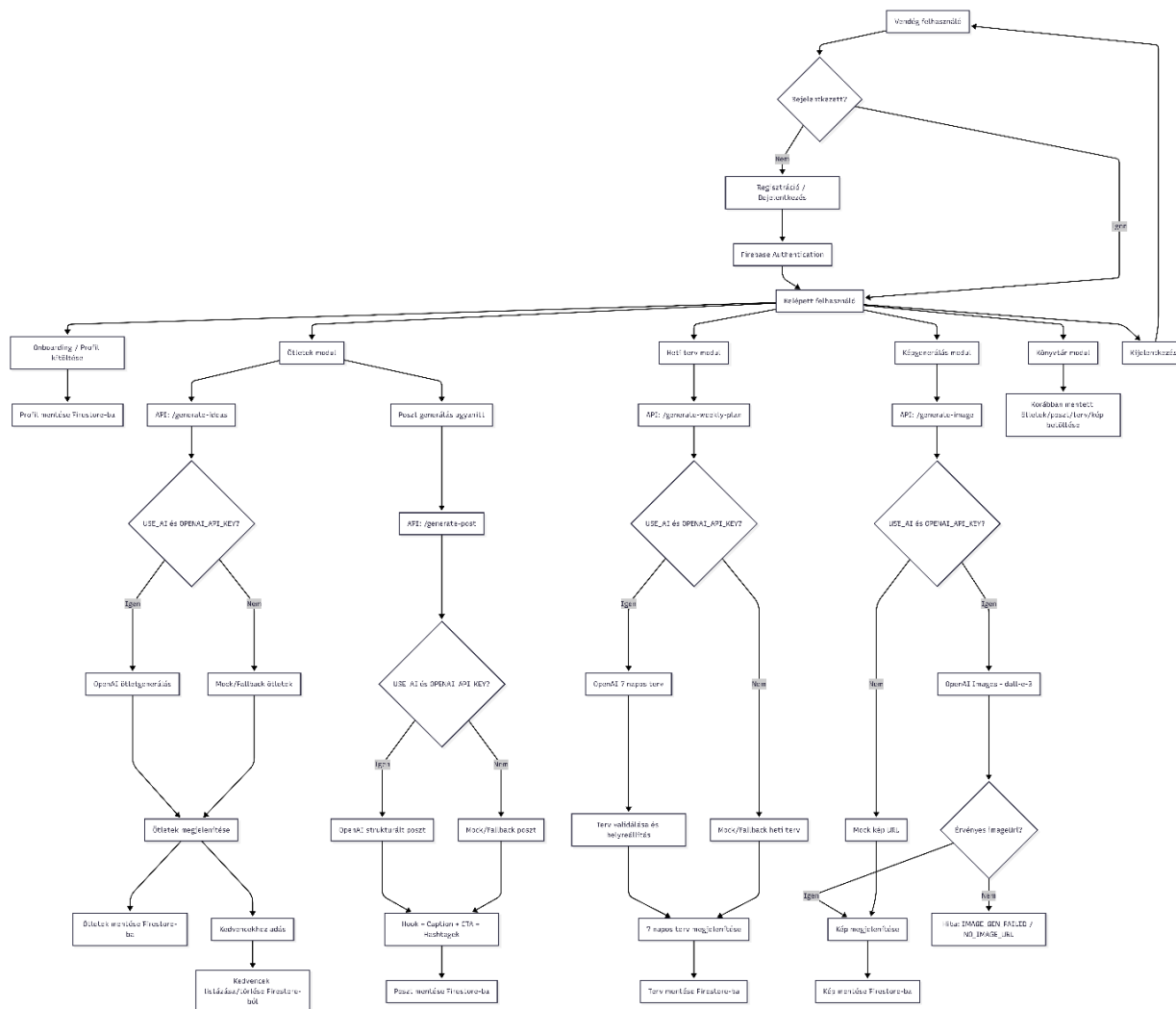
2.3. Ocoya

Az Ocoya a mesterséges intelligencia által támogatott szövegírást ötvözi a grafikus megjelenítéssel és szerkesztéssel és a beépített tartalomütemező funkcióval. Erőssége, hogy a platformon a tervezéstől a publikálásig egy folyamaton keresztül végig lehet menni. Nincs ingyenesen elérhető verziója, ami elég nagy hátrány egy ilyen alkalmazásnál. A mesterséges intelligencia által generált szövegtartalmak sablonosak, és hiányzik a mélyebb, profilalapú személyre szabás lehetősége. A magyar nyelvi támogatás minimális, a felhasználói felület a funkciók zsúfoltsága miatt kevésbé átlátható és nehezen kezelhető.

2.4. Simplified

A Simplified egy multifunkcionális kreatív alkalmazás, amely a design, videóvágás és szövegírás területeit fedi le mesterséges intelligencia segítségével. Korlátozott ingyenes csomaggal rendelkezik, komplexitása miatt széleskörű igényeket képes kiszolgálni. A platform legnagyobb gyengesége a fókuszátlanság; a széles funkcionalitás miatt a közösségi média specifikus íróeszközei felületesek maradnak. Nem támogatja a profilalapú, iparág-specifikus tartalomgyártást, az ingyenes keretrendszer pedig rendkívül gyorsan kimerül, ami ellehetetleníti a rendszer alapos tesztelését vagy kisebb vállalkozások általi tartós használatát.

3. FUNKCIONÁLIS SPECIFIKÁCIÓ



3.1. ábra: A SocialBoost AI rendszer fő felhasználói folyamatainak funkcionális folyamatábrája

3.1. Bevezetés

Az alkalmazás célja, hogy a felhasználó számára gyors, egyszerű és érthető módon tudjon profilra, márkára és célközönségre specifikálva közösségi média tartalmat készíteni, valamint poszthoz kapcsolódó marketing-képeket elkészíteni. A felhasználói adatok kezelése és regisztráció, bejelentkezés Firebase Authentication és Cloud Firestore szolgáltatásokkal valósul meg. Az alkalmazás úgy lett kialakítva, hogy a felhasználó egyetlen felületen végig tudja vinni

a posztkészítési folyamatot az üzleti profil beállításától a generált tartalmak elkészítéséig és mentéséig.

Az alkalmazás autentikációja Firebase Authentication szolgáltatására épül, e-mail és jelszó alapú hitelesítéssel. A rendszer csak bejelentkezett avagy regisztrált felhasználók számára teszi elérhetővé az alkalmazás tartalomgeneráló és mentési funkcióit. Az adatok megfelelő tárolásáért, titkosításukért és biztonságos bejelentkezésért a Firebase Authentication felel.

A regisztráció során a felhasználó egy új fiókot hoz létre e-mail és jelszóval. Sikeres regisztráció után a rendszer egy hitelesített munkamentet indít, így a felhasználó hozzáfér a védett oldalakhoz. A regisztráció céljai egyedi felhasználói fiók létrehozása egyedi üzletprofil adatokkal, és a felhasználó más eszközökről később, vagy akár az alkalmazásból való kijelentkezés és újbóli bejelentkezés után is hozzáfér az adataihoz a korábban generált és mentett tartalmakkal együtt.

Sikeres regisztráció után a rendszer átirányít az üzletprofil oldalra, és a profil létrehozása során Firebase UID generálódik a háttérben, amellyel bármelyik felhasználó könnyen azonosítható. A mellékelt képen látszódik a Firebase projekt konzolon a regisztráció.

Amennyiben a felhasználó kijelentkezik, avagy bezárja, internet nélkül használja az alkalmazást, később bejelentkezésre lesz szüksége az újbóli belépéshez és védett oldalak eléréséhez. Ezt a korábban már regisztrált e-mail címének és jelszavának beírásával a bejelentkezés gombra kattintva teheti meg. Hibás adatok vagy sikertelen azonosítás esetén az alkalmazás nem engedi bejelentkezni a felhasználót. Amennyiben a felhasználó helyes adatokkal tölti ki a mezőket, a bejelentkezés megvalósul, a felhasználó a posztötletek oldalra kerül és az betöltődik az esetlegesen már korábban generált posztokkal és a posztgeneráláshoz szükséges kitöltött adatokkal.

A rendszer autentikációs rétege két egymást kiegészítő komponensre épül. A 3.4-es ábrán látható kód feladata a Firebase SDK környezet inicializálása, az autentikációs szolgáltatás és adatbázis kapcsolat központi létrehozása. Ezzel szemben a 3.5-ös ábrán a kód a hitelesítés üzleti logikáját valósítja meg, a Firebase Authentication függvényeinek alkalmazásával. A szolgáltatás kezeli a regisztrációs, bejelentkezési és kijelentkezési műveleteket.

A rendszer útvonalvédelmet alkalmaz. Az ötletek, könyvtár, tervező, képgenerálás és onboarding oldalak kizárólag hitelesített felhasználók számára érhetőek el. Nem hitelesített felhasználó esetén a rendszer nem engedi az oldal megnyitását.

A felhasználó a jobb oldalon lévő menüből a kijelző alján lévő kilépés gombra kattintva szüntetheti meg az aktív munkamenetet. Ezután kizárólag a nyilvános és vendég felhasználók

számára is elérhető landing oldal jelenik meg, a védett oldalak nem. Ha az alkalmazást újból használni szeretné, be kell jelentkeznie a korábban beregisztrált fiókjával.

3.2. Autentikáció

Az alkalmazás autentikációja Firebase Authentication szolgáltatására épül, e-mail és jelszó alapú hitelesítéssel. A rendszer csak bejelentkezett avagy regisztrált felhasználók számára teszi elérhetővé az alkalmazás tartalomgeneráló és mentési funkcióit. Az adatok megfelelő tárolásáért, titkosításukért és biztonságos bejelentkezésért a Firebase Authentication felel.

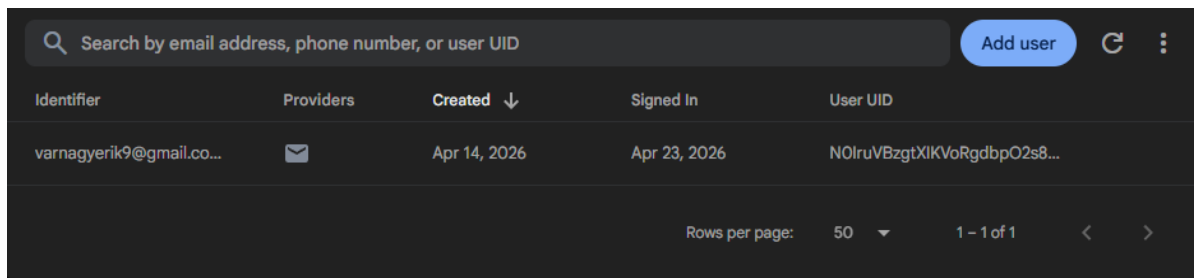
3.2.1. Regisztráció

A regisztráció során a felhasználó egy új fiókot hoz létre e-mail és jelszóval. Sikeres regisztráció után a rendszer egy hitelesített munkamentet indít, így a felhasználó hozzáfér a védett oldalakhoz. A regisztráció céljai egyedi felhasználói fiók létrehozása egyedi üzletprofil adatokkal, és a felhasználó más eszközökről később, vagy akár az alkalmazásból való kijelentkezés és újbóli bejelentkezés után is hozzáfér az adataihoz a korábban generált és mentett tartalmakkal együtt.

The screenshot shows the user interface of the SocialBoost AI application. At the top, there is a header with the text "Üdv a SocialBoost AI-ban" and a subtitle "Gyorsabb tervezés, jobb posztok, egy helyen." To the right of the header are two links: "Home" and "Bejelentkezés". Below the header, there are two main sections. The left section is titled "SOCIALBOOST AI" and "Tervezz, generálj, posztolj gyorsabban." It contains a description: "Egy helyen kezelheted a posztötleteket, a tartalomtervet és az inspirációs könyvtáradat." and a list of features: "• Okos ötletgenerálás célközönségre szabva", "• Menthető posztok és kedvencek", and "• Heti tartalomterv másodpercek alatt". The right section is titled "Belépés" and contains the text "Jelentkezz be, vagy hozz létre fiókot." Below this text are two input fields: "Email" and "Jelszó". At the bottom of the right section are two buttons: "Belépés" and "Regisztráció".

3.2 ábra : Regisztráció és bejelentkezés oldal

Sikeres regisztráció után a rendszer átirányít az üzletprofil oldalra, és a profil létrehozása során Firebase UID generálódik a háttérben, amellyel bármelyik felhasználó könnyen azonosítható. A mellékelt képen látszódik a Firebase projekt konzolon a regisztráció.



Identifier	Providers	Created ↓	Signed In	User UID
varnagyerik9@gmail.co...		Apr 14, 2026	Apr 23, 2026	NOlruVBzgtXIKVoRgdbpO2s8...

Rows per page: 50 ▼ 1 – 1 of 1 < >

3.3 ábra: Firebase authentication által létrehozott hitelesített fiók

3.2.2. Autentikált bejelentkezés

Amennyiben a felhasználó kijelentkezik, avagy bezárja, internet nélkül használja az alkalmazást, később bejelentkezésre lesz szüksége az újbóli belépéshez és védett oldalak eléréséhez. Ezt a korábban már regisztrált e-mail címének és jelszavának beírásával a bejelentkezés gombra kattintva teheti meg. Hibás adatok vagy sikertelen azonosítás esetén az alkalmazás nem engedi bejelentkezni a felhasználót. Amennyiben a felhasználó helyes adatokkal tölti ki a mezőket, a bejelentkezés megvalósul, a felhasználó a posztötlek oldalra kerül és az betöltődik az esetlegesen már korábban generált posztokkal és a posztgeneráláshoz szükséges kitöltött adatokkal.

```
import { initializeApp } from 'firebase/app';
import { getAuth } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';
import { environment } from './environment';

export const firebaseApp = initializeApp(environment.firebase);
export const auth = getAuth(firebaseApp);
export const db = getFirestore(firebaseApp);
```

3.4 ábra: Firebase szolgáltatások inicializálása Angular környezetben

A rendszer autentikációs rétege két egymást kiegészítő komponensre épül. A 3.4-es ábrán látható kód feladata a Firebase SDK környezet inicializálása, az autentikációs szolgáltatás és adatbázis kapcsolat központi létrehozása. Ezzel szemben a 3.5-ös ábrán a kód a hitelesítés üzleti logikáját valósítja meg, a Firebase Authentication függvényeinek alkalmazásával. A szolgáltatás kezeli a regisztrációs, bejelentkezési és kijelentkezési műveleteket.

```

import { Injectable } from '@angular/core';
import { auth } from '../firebase';
import {
  onAuthStateChanged,
  signInWithEmailAndPassword,
  createUserWithEmailAndPassword,
  signOut,
  User,
} from 'firebase/auth';

@Injectable({ providedIn: 'root' })
export class AuthService {
  user: User | null = null;

  constructor() {
    onAuthStateChanged(auth, (u) => {
      this.user = u;
    });
  }

  async register(email: string, password: string) {
    await createUserWithEmailAndPassword(auth, email, password);
  }

  async login(email: string, password: string) {
    await signInWithEmailAndPassword(auth, email, password);
  }

  async logout() {
    await signOut(auth);
  }
}

```

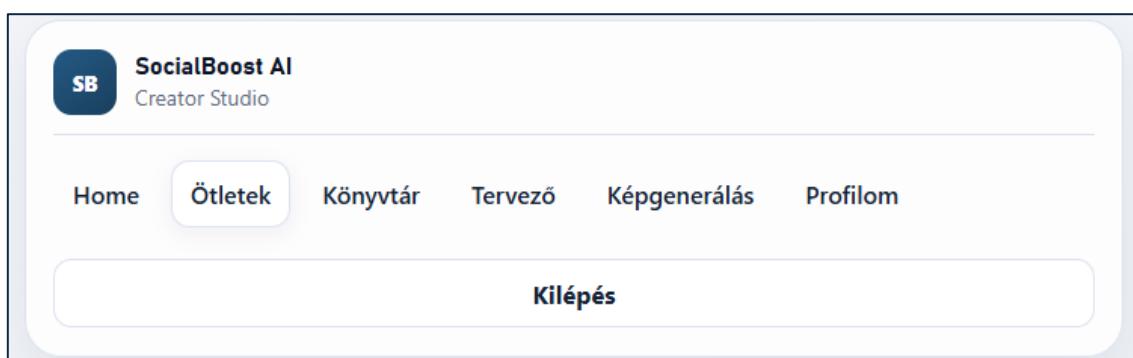
3.5 ábra: Az autentikációs szolgáltatások implementációja

3.2.3. Jogosultságkezelés

A rendszer útvonalvédelmet alkalmaz. Az ötletek, könyvtár, tervező, képgenerálás és onboarding oldalak kizárólag hitelesített felhasználók számára érhetőek el. Nem hitelesített felhasználó esetén a rendszer nem engedi az oldal megnyitását.

3.2.4. Kijelentkezés

A felhasználó a jobb oldalon lévő menüből a kijelző alján lévő kilépés gombra kattintva szüntetheti meg az aktív munkamenetet. Ezután kizárólag a nyilvános és vendég felhasználók számára is elérhető landing oldal jelenik meg, a védett oldalak nem. Ha az alkalmazást újból használni szeretné, be kell jelentkeznie a korábban beregisztrált fiókjával.



3.6 ábra: A kilépés gomb a menü mobil nézetben

3.3. Az alkalmazás fő funkciói

Az alkalmazás fő funkcióinak használatához bejelentkezés szükséges. A rendszer moduláris felépítésű: minden fő funkció külön oldalhoz tartozik, de közös profil és mentési logikát használnak.

3.3.1. Kezdőoldal

A kezdőoldalon az alkalmazás fő funkciói, példa generált poszt részletesen, illetve az alkalmazás előnyei szövegesen feltüntetve. Rövid leírás arról, hogyan működik az alkalmazásban egy ötlet és poszt generálása a belépéstől a generálásig. Ez az oldal nem bejelentkezett felhasználók számára egy tájékoztató oldal, bejelentkezve pedig home oldal ami az alkalmazás navigációs panelén keresztül érhető el.

3.3.2. Üzletprofil oldal

Az üzletprofil oldal célja, hogy a mesterséges intelligencia által generált tartalom teljesen személyes igényekre legyen szabva. Ez az oldal fogadja a frissen regisztrált felhasználókat, hiszen a személyre szabott tartalomhoz elengedhetetlenek bizonyos adatok.

The screenshot shows the 'Üzletprofil' (Business Profile) page in the SocialBoost AI Creator Studio. The page is part of a dashboard and is titled 'Dashboard' with the subtitle 'Gyorsabb tervezés, jobb posztok, egy helyen.' (Faster planning, better posts, one place). The main content area is titled 'Üzletprofil' and includes a progress bar indicating 'Lépés 1 / 3' (Step 1 / 3). Below the progress bar, there is a section titled 'Alapok' (Basics) with four input fields for user information: 'Üzletág (pl. kozmetika, fotós)' (Business type (e.g., cosmetics, photographer)), 'Konkrét téma vagy célközönség (pl. fiatal anyukák, divat iránt érdeklődők)' (Specific topic or target audience (e.g., young mothers, fashion enthusiasts)), 'Lokáció (pl. Budapest, Debrecen vagy országos)' (Location (e.g., Budapest, Debrecen or nationwide)), and 'Korosztály (pl. 18-24, 25-34)' (Age group (e.g., 18-24, 25-34)). There are 'Vissza' (Back) and 'Tovább' (Next) buttons at the bottom of the form. On the left side, there is a sidebar with navigation links: 'Home', 'Ötletek' (Ideas), 'Könyvtár' (Library), 'Tervező' (Designer), 'Képgenerálás' (Image generation), and 'Profilom' (My profile). At the bottom of the sidebar is a 'Kilépés' (Logout) button.

3.7 ábra: Üzletprofil oldal első állapota

Elsőként az alapok fülénél meg kell adni üzletágat, konkrét témát vagy célközönséget, lokációt, korosztályt. Az adatok kitöltése után a tovább gombra kattintva lenyíló menüből kiválasztható a platform és a konkrét cél amit a poszt segítségével elérni hivatott a felhasználó. Ezután jön egy márkahang oldal, ahol a posztolandó szöveg hangvételét lehet beállítani, és ellenőrizni az adatokat mentés előtt. Majd a mentés és továbblépés gombra kattintva az oldal átnavigál a posztötletek oldalra. A profiladatok Cloud Firestore-ba mentődnek, és a generálási végpontoknak ez kerül átadásra kontextusként. Később bármikor módosítható.

SB

SocialBoost AI

Creator Studio

Home

Ötletek

Könyvtár

Tervező

Képgenerálás

Profilom

Kilépés

Dashboard

Gyorsabb tervezés, jobb posztok, egy helyen.

Üzletprofil

Állítsd be az üzletadatokat, platformot és márkahangot a személyre szabott ötletekhez.

Lépés 2 / 3

Platform és cél

Facebook

Érdeklődők gyűjtése

Vissza

Tovább

3.8 ábra : Üzletprofil oldal második állapota

SB

SocialBoost AI

Creator Studio

Home

Ötletek

Könyvtár

Tervező

Képgenerálás

Profilom

Kilépés

Dashboard

Gyorsabb tervezés, jobb posztok, egy helyen.

Üzletprofil

Állítsd be az üzletadatokat, platformot és márkahangot a személyre szabott ötletekhez.

Lépés 3 / 3

Márkahang

Barátságos

Ellenőrzés mentés előtt:

- **Üzletág:** kőrmös
- **Célközönség:** fiatal lányok
- **Lokáció:** Budapest
- **Korosztály:** 16-26
- **Platform:** facebook
- **Fókusz:** lead
- **Hang:** friendly

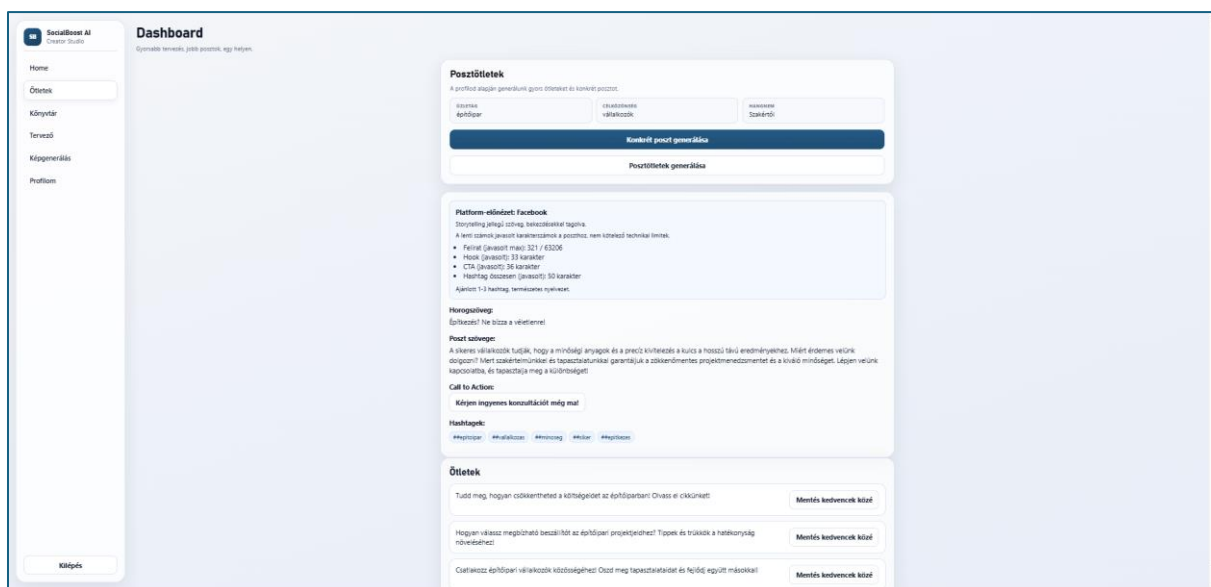
Vissza

Mentés és továbblépés

3.9 ábra: Üzletprofil harmadik állapota

3.3.3. Rövid posztötlet és komplett posztszöveg generáló oldal

A felhasználó megadott profiladatai segítségével lehet ezen az oldalon egy rövid posztötletet, és komplett posztstruktúrát részletesen generálni. A fehér posztötletek generálása gombra kattintva 3 generált rövid posztötlet készül el, melyek mentésére is lehetőség van. A kék konkrét poszt generálása gombra kattintva komplett posztstruktúra készül el részletesen platformfüggő tippekkel, maximálisan javasolt karakter és hashtagszámokkal, horogszöveggel, posztszöveggel, cselekvésre ösztönző egymondatos szöveggel, hashtagekkel. Ezen funkciók jelentősen gyorsítják a posztkészítést, hisz egy másolással és beillesztéssel a poszt lényegi részét el tudja készíteni a felhasználó.

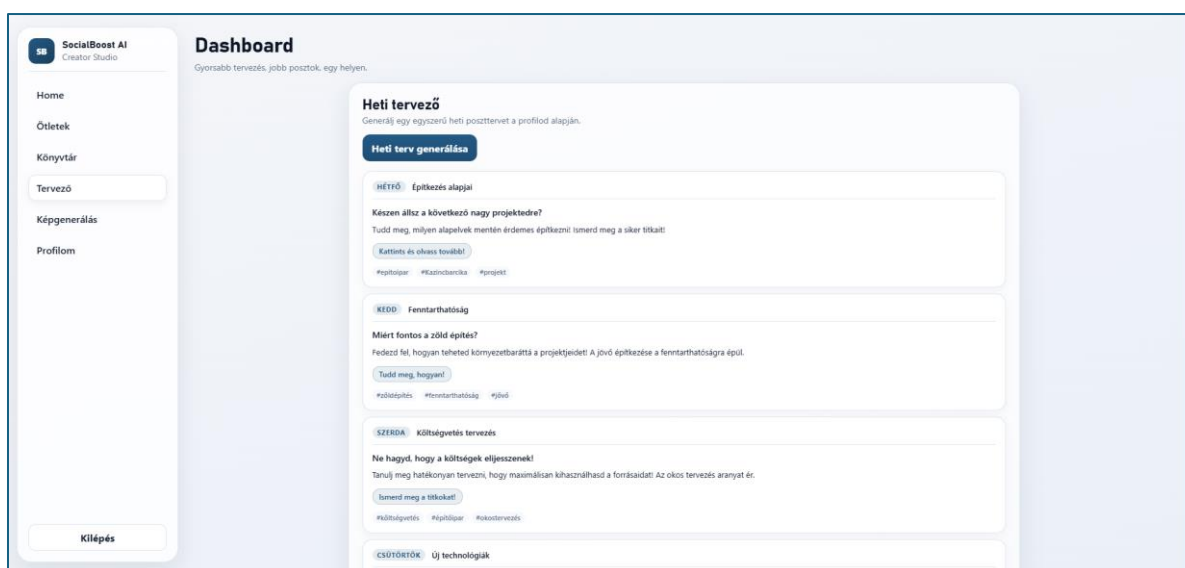


3.10 Posztötlet és strukturált poszt generáló modul

3.3.4. Heti tervező oldal

A közösségi médiára posztoló kis- és középvállalkozások oldalán nem azért inaktívak mert nincs mondanivalójuk, hanem mert nincs előretervezett rendszerük, és nincs kapacitásuk minden egyes nap leülni a számítógép elé és kitalálni egy komplett saját posztot. A heti tervező funkció képes a profiladatok alapján egy 7 napra bontott tartalomtervet készíteni. A cél, hogy a felhasználó rendszeres és strukturált módon tudjon posztolni, és az inaktivitás mértéke csökkenjen. A posztok napi bontásban részletesen generálódnak. A poszt részletei lebontva nap,

téma, horogszöveg, cselekvésre ösztönző szöveg, hashtag-ek. A funkció időt takarít meg, mivel a folyamatos jelenlét támogatása mellett a felhasználó előre elkészített kommunikációs tervből dolgozhat. A funkciót a felhasználó az oldalra való navigálás után a heti terv generálása gombra kattintva tudja használni. Ha esetleg nem tetszenének neki azok az ötletek, a gomb újbóli kattintásával új tervet tud elkészíteni. A tervezhetőség nagy üzleti értéket biztosít: jobb követői élmény, kiszámíthatóbb kommunikáció, a posztoló számára kevesebb erőforrás igénybevétel.

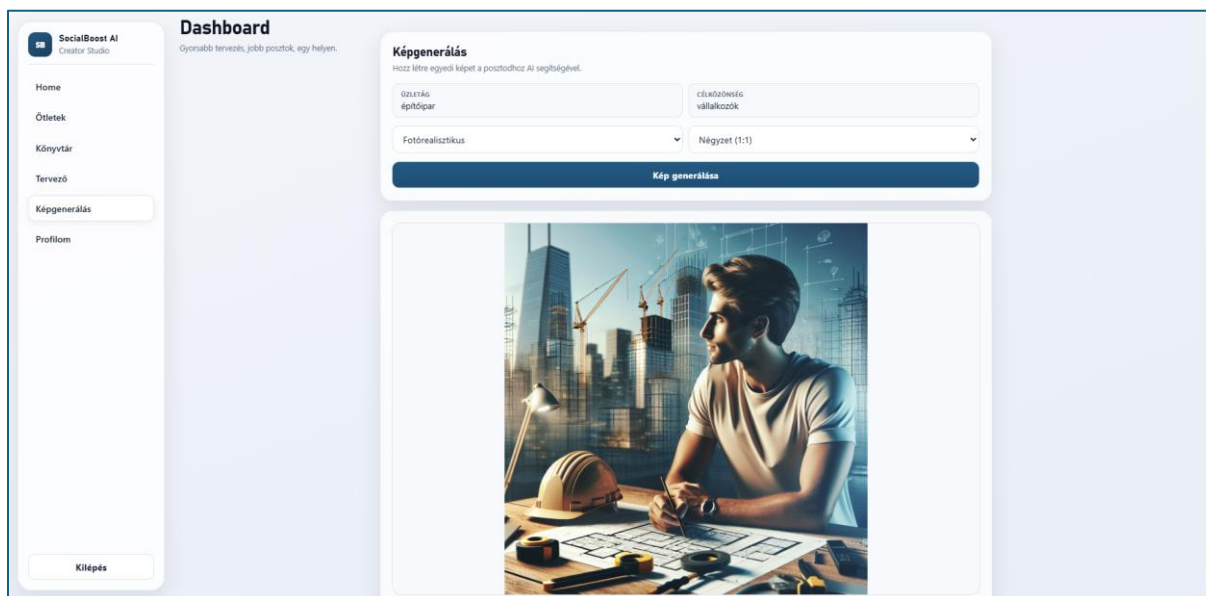


3.11 ábra: Heti tervező oldal és generált heti terv

3.3.5. Képgenerálás oldal

A felhasználó számára gyakori akadály, hogy van jó szöveg, de nincs megfelelő kép, illetve ötlete sincs milyen képet kellene használni egy közösségi médiás poszthoz, illetve hogyan kellene egyáltalán elkészíteni. Az alábbi funkció erre hivatott megoldást találni. A felhasználó a képgenerálás oldalon miután ellenőrizte az ott megjelenő profiladatait, kettő lenyíló listából választhat egy-egy tulajdonságot a kép generálása előtt. Az egyikben a kép stílusát, mint realiztikus és minimalista, a másikon a képarányt. A választás után a kép generálása gombra kattintva elkészül a kép, és az alatta lévő kép megnyitása gombra kattintva lehet megnyitni és akár letölteni. Előnye a funkciónak még, hogy a választási lehetőségek miatt több stílus is kipróbálható, és platformfüggetlen több képarány. A generálás másodperceken belül történik, ha esetleg a felhasználónak nem tetszene, a gomb újbóli kattintásával bármikor tud egy másik

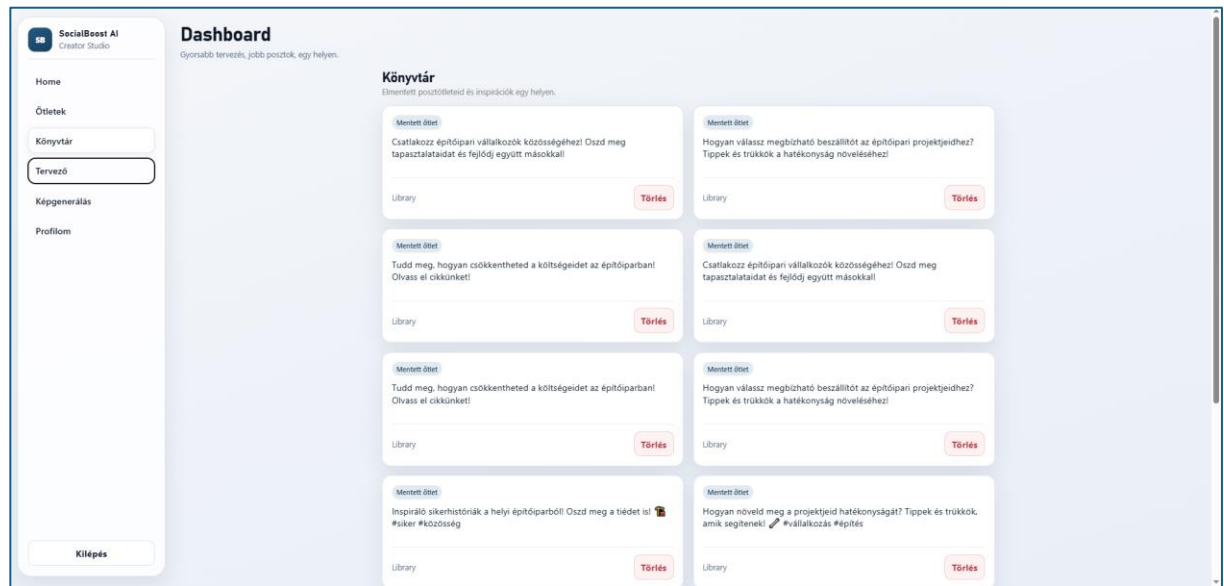
képet generálni. A szöveg és a kép együtt egy teljes posztot eredményeznek, ezek a funkciók együtt kisvállalkozók számára kifejezetten hasznosak, mivel csökkentik a külső költségigényt és esetlegesen más alkalmazások használatának szükségét.



3.3 Képgenerálás oldal, generált fotóval

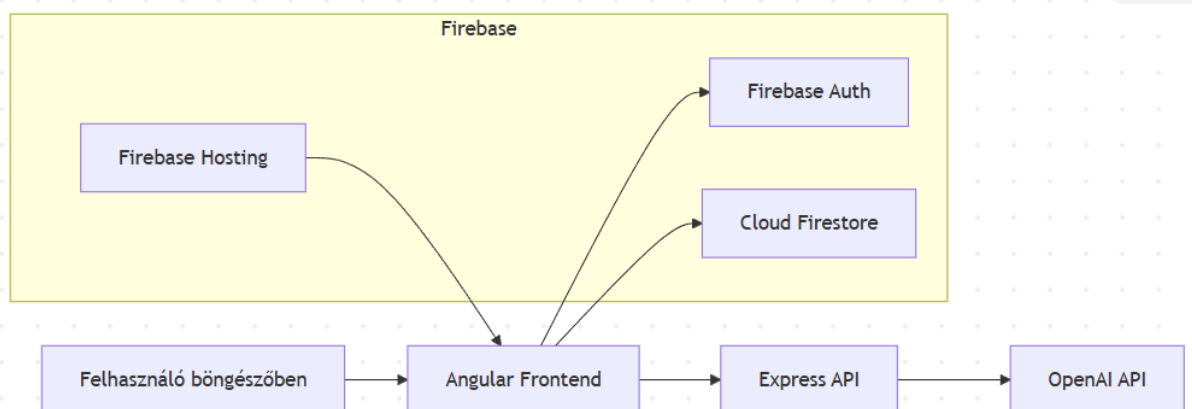
3.3.6. Könyvtár oldal

Az alkalmazásban a felhasználónak lehetősége van a kedvenc ötleteit elmenteni, a posztötletek oldalon a generálás után. Így a felhasználónak lehetősége van a későbbi posztok visszakeresésére is, hogy a generált tartalomból később is tudjon válogatni. Az esetlegesen elmentés után mégsem fontos posztokat tudja törölni a piros feliratú törlés gombra kattintva.



3.3 Könyvtár oldal elmentett ötletekkel

4 BELSŐ FELÉPÍTÉS



4.1 ábra: Egyszerű architektúra-ábra

A SocialBoost AI webalkalmazás Angular alapokon készült, standalone komponens-architektúrával. A megközelítés lényege, hogy az alkalmazás modulok helyett közvetlenül komponensekre és szolgáltatásokra épül, így a kódbázis egyszerűbben szervezhető és könnyebben karbantartható. A felület több, üzleti funkció szerint elkülönített oldalból áll, amelyeket a router dinamikusán tölt be. A kliensoldali működés három fő rétegre bontható: megjelenítési réteg (komponensek), üzleti logikai réteg (service-ek), valamint adatkezelési/integrációs réteg (Firebase és API kommunikáció).

4.1. Komponensek

Az alkalmazás komponensei két nagy csoportba sorolhatók.

1. Oldal (Page) komponensek

Ezek közvetlenül útvonalhoz kötött, navigálható képernyők.

- App komponens

A rendszer gyökérkomponense, amely a fő layoutot és a route-alapú tartalommegjelenítést kezeli.

- Home oldal

A kezdőfelület, amely bemutatja az alkalmazás fő funkcióit.

- Login oldal
A regisztrációs és bejelentkezési folyamatok belépési pontja.
- Onboarding oldal
A felhasználói üzleti profil felvételét és módosítását biztosítja.
- Ideas oldal
Posztötlet- és posztgenerálási funkciókat biztosít.
- Planner oldal
Heti tartalomterv-generálásra és tervezésre szolgál.
- Image-Gen oldal
Marketingképek generálását és előnézetét biztosítja.
- Library oldal
A korábban mentett generált tartalmak áttekintésére szolgál.

2. Beágyazott (UI/Feature) komponensek

Ezek az oldalakon belüli részfunkciók megvalósításáért felelnek.

- Űrlap komponensek
Például ötletgeneráló, képgeneráló és onboarding űrlapok.
- Fejléc és információs komponensek
Oldalszintű kontextus, státusz és magyarázó tartalmak megjelenítése.
- Eredmény komponensek
Generált ötletek, posztok, képek és heti terv elemek megjelenítése.
- Lista- és elemkomponensek
Könyvtár, kedvencek, tervek és kártyák strukturált listázása.

A komponensfelépítés előnye, hogy az egyes funkciók kis, jól újrafelhasználható egységekre bonthatók, ezáltal gyorsabb a fejlesztés és átláthatóbb a hibakeresés.

4.2 Szolgáltatások (Services)

Az alkalmazás üzleti logikáját és adatforgalmát service osztályok kezelik. Ezek dependency injection segítségével kerülnek a komponensekbe.

- AuthService

A felhasználói hitelesítéshez kapcsolódó műveleteket valósítja meg (regisztráció, bejelentkezés, kijelentkezés, auth state figyelés).

- ProfileService

A felhasználó üzleti profiladatainak mentését és lekérését kezeli.

- PlannerService

A heti tartalomterv-generálási logikát és API kommunikációt valósítja meg.

- GeneratedContentService

A generált ötletek, posztok, heti terv és képadatok Firestore-mentését és visszatöltését biztosítja.

- FavoritesService

A kedvenc elemek létrehozását, listázását és törlését kezeli.

A service-réteg fő előnye, hogy a felület (UI) és az üzleti logika elválik egymástól, így a rendszer jobban tesztelhető és könnyebben továbbfejleszthető.

4.3 Adatmodellek és típusok

Az alkalmazás TypeScript típusokkal írja le a használt adatstruktúrákat, ami növeli a kód megbízhatóságát és csökkenti a futásidejű hibák valószínűségét.

Főbb kliensoldali modellek:

- PlanItem

A heti terv egy napját reprezentáló adatstruktúra.

- Idea

Egy generált posztötlet típusleírása.

- GeneratedIdeasDoc

Mentett ötletek és opcionális generált poszt dokumentumstruktúrája.

- GeneratedPlanDoc

Mentett heti terv dokumentumstruktúrája.

- GeneratedImageDoc

Mentett képgenerálási eredmény struktúrája.

- FavoriteIdea

Kedvencként mentett elem típusleírása.

Szerveroldalon a bemenetek validálása Zod sémákkal történik, így a backend csak szabályos és elvárt szerkezetű kéréseket dolgoz fel.

4.4 Routing és jogosultságkezelés

Az alkalmazás navigációs logikáját az Angular Router valósítja meg, amely a kliensoldali útvonalkezelés központi eleme. A routing célja, hogy a felhasználó egyoldalas (SPA) működés mellett, teljes oldalfrissítés nélkül tudjon mozogni a funkcionális nézetek között. A rendszerben az útvonalak egységesen route-konfiguráció alapján vannak meghatározva, így a különböző oldalak elérése URL-szinten is egyértelműen leképezhető. A route-definíciók szerint a bejelentkezéshez kötött funkciók és a nyilvánosan elérhető nézetek elkülönülnek, ami egyszerre szolgálja a biztonságot és az átlátható felhasználói élményt. A fő felhasználói nézetek, mint az ötletgenerálás, a könyvtár, a tervező, a képgenerálás és az onboarding, külön útvonalakon keresztül érhetők el, míg a belépési folyamat a login útvonalon történik.

A rendszer route-kezelésének egyik fontos sajátossága, hogy a komponensek dinamikusan, lazy módon töltődnek be (loadComponent), ezért az alkalmazás indulási ideje alacsonyabb, és csak azok a nézeti egységek kerülnek betöltésre, amelyekre a felhasználó ténylegesen navigál. Ez nemcsak teljesítmény-optimalizálási előnyt jelent, hanem skálázhatósági szempontból is kedvező, mert a későbbi bővítések során új funkciók külön route-ként beilleszthetők a meglévő struktúra megbontása nélkül. A gyakorlatban ez azt eredményezi, hogy a rendszer jól karbantartható marad akkor is, ha a jövőben további tartalommodulok, például új generálási nézetek vagy analitikai oldalak kerülnek bevezetésre.

A jogosultságkezelés route-szinten az auth guard mechanizmuson keresztül valósul meg. A védett oldalak csak hitelesített felhasználó számára érhetők el, a guard a navigáció előtt ellenőrzi a hitelesítési állapotot, és jogosulatlan hozzáférés esetén a felhasználót a bejelentkezési oldalra irányítja vissza. Ennek különös jelentősége van az alkalmazásban, mert a védett felületek személyes profiladatokat, mentett generált tartalmakat és felhasználói erőforrásokat kezelnek. A route-védelem tehát nem csupán technikai megoldás, hanem adatvédelmi és működésbiztonsági követelmény is, amely biztosítja, hogy a felhasználó csak a saját munkaterületéhez kapcsolódó nézeteket és műveleteket érhesse el.

A routing felhasználói élmény szempontjából is meghatározó: a navigáció konzisztens, a fő funkciók logikusan csoportosított útvonalakon keresztül érhetők el, az ismeretlen vagy hibás URL-ek pedig egy alapértelmezett útvonalra kerülnek visszairányításra. Ez csökkenti az elakadási pontokat, és biztosítja, hogy a felhasználó bármely állapotból vissza tudjon térni egy értelmezhető felületre. Összességében a routing megoldás a rendszer belső architektúrájának egyik kulcseleme: egyszerre támogatja a teljesítményt, a biztonságot, a bővíthetőséget és a jól strukturált felhasználói folyamatokat.

4.5 Kliens-szerver integráció

Az alkalmazás hibrid architektúrát használ:

1. Firebase szolgáltatások
2. Hitelesítés és tartós adattárolás (felhasználói profil, mentett generált tartalmak, kedvencek).
3. Egyedi backend API
4. AI-alapú generálási feladatok kezelése (ötlet, poszt, heti terv, kép).

Fő API végpontok:

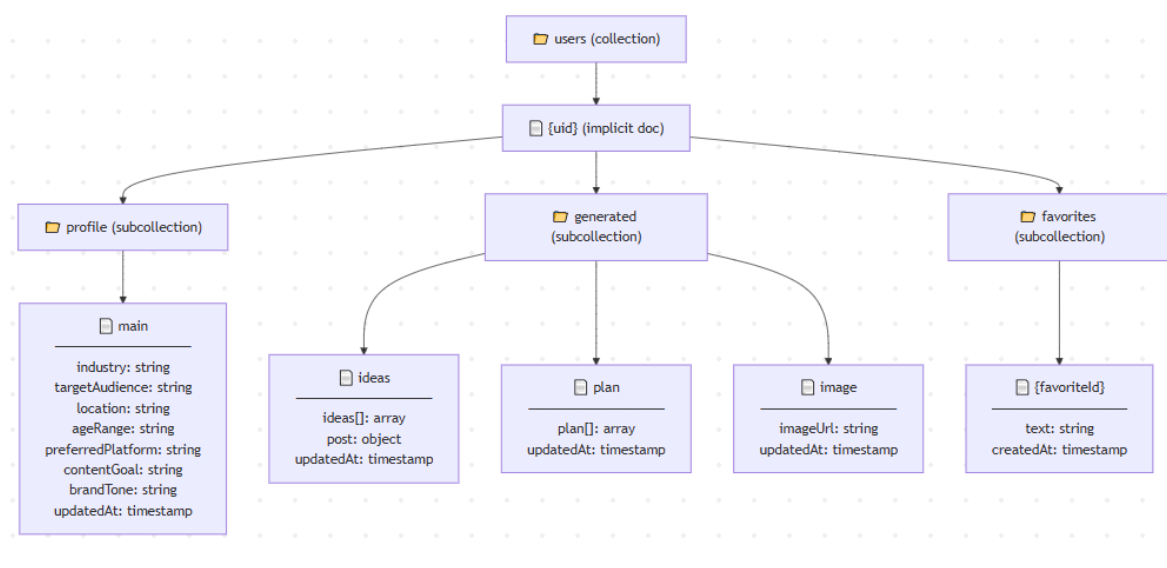
- /generate-ideas
- /generate-post
- /generate-weekly-plan
- /generate-image
- /health

A kliensoldal service rétege HTTP kérésekkel hívja ezeket a végpontokat, majd a kapott eredményeket megjeleníti és szükség esetén Firestore-ba menti.

4.6 Firestore-ban tárolt adatok

A rendszer tartós adatkezelését a Firebase Firestore biztosítja, ahol a felhasználóhoz kapcsolódó adatok logikailag elkülönített dokumentumokban és részgyűjteményekben tárolódnak. A felhasználói üzleti profil a `users/{uid}/profile/main` dokumentumban helyezkedik el, és olyan mezőket tartalmaz, mint az `industry`, `targetAudience`, `location`, `ageRange`, `preferredPlatform`, `contentGoal` és `brandTone`. A generált tartalmak a `users/{uid}/generated` útvonal alatt külön dokumentumokban kerülnek mentésre: az `ideas` dokumentum a generált ötletlistát és az opcionális generált posztot tárolja (`GeneratedIdeasDoc`), a `plan` dokumentum a heti tartalomtervet (`GeneratedPlanDoc`), míg az `image` dokumentum a generált kép elérési útját (`GeneratedImageDoc`) tartalmazza. Ezekhez

minden esetben időbélyeg is társul az updatedAt mezőben. A kedvencként elmentett elemek külön részgyűjteményben, a users/{uid}/favorites útvonalon helyezkednek el, ahol minden dokumentum egy önálló FavoriteIdea rekordként tárolódik, tipikusan text és createdAt mezőkkel. Ez a struktúra jól támogatja a felhasználónkénti adatszeperációt, az egyszerű lekérdezhetőséget és a generált tartalmak későbbi visszatöltését.



4.2 ábra : Firestore adatbázis struktúra

4.7 Belső működési összesség

A rendszer belső felépítése komponensalapú és rétegzett. A page komponensek a felhasználói folyamatot fedik le, a service-ek tartalmazzák az üzleti logikát, míg a Firebase és a backend API biztosítják az adattárolást és a generálási funkciókat. Ez az architektúra jól támogatja a skálázhatóságot, mert új funkciók külön komponensben és szolgáltatásban hozzáadhatók anélkül, hogy a meglévő rendszerstruktúrát jelentősen módosítani kellene.

5 SAJÁT API BEMUTATÁSA

A rendszer szerveroldali komponense egy egyedi, Node.js alapú REST API, amelynek elsődleges feladata a tartalomgenerálási folyamatok kiszolgálása és a kliensalkalmazás üzleti logikájának támogatása. Az API nem általános célú backendként működik, hanem kifejezetten a SocialBoost AI funkcióihoz készült: ötletgenerálás, posztgenerálás, heti tartalomterv készítése és képgenerálás.

5.1. Az API szerepe a rendszerben

Az API a frontend és az AI-szolgáltatás közötti köztes réteggént működik. Ez a réteg az alábbi előnyöket biztosítja:

1. A kliensoldal mentesül az AI-hívások közvetlen kezelésétől.
2. A bemenetek szerveroldali validációja egységesen történik.
3. A külső szolgáltatási hibák kezelése kontrollált módon valósul meg.
4. A fallback logika miatt részleges szolgáltatáskiesés esetén is marad működő kimenet.

5.2. Technológiai alapok

Az API az alábbi komponensekre épül:

1. Express.js: HTTP végpontok és middleware-kezelés.
2. Zod: bemeneti adatok típusos és szabályalapú validációja.
3. OpenAI SDK: szöveges és képi generálási műveletek.
4. dotenv: környezeti változók betöltése.
5. cors: cross-origin kérések kezelése.

A szerver JSON-kéréseket fogad, naplózza a beérkező hívásokat, és minden végpontnál strukturált hibaválaszt ad.

5.2.1. API-védelmi megoldások

Az API végpontjait négy egymást kiegészítő biztonsági réteg védi:

1. **Firestore ID token ellenőrzés:** minden védett kéréshez a kliensoldal Authorization: Bearer <Firestore ID token> fejléccet küld; a szerver a Firestore Admin SDK verifyIdToken() függvényével ellenőrzi az aláírást és a lejáratot, érvénytelen token esetén 401-es hibával utasítja vissza a kérést.
2. **Bearer token (Authorization fejléc):** a szabványos HTTP Authorization: Ha hiányzik vagy hibás a Bearer token, a szerver 401-et ad vissza ("Missing Bearer token" vagy "Invalid Firestore ID token").
3. **CORS allowlist:** a CORS köztes réteg kizárólag az engedélyezési listán szereplő frontend origin(ek)e)t fogadja el; idegen domain-ről érkező böngészős kéréseket a szerver tiltó CORS-válasszal utasít vissza. A middleware nem engedi tovább a kérést, ha a fejléc nem szabványos.
4. **Rate limit:** A projekt saját egyszerű rate limit logikát használ. Ez userId vagy IP alapján számolja a kéréseket, és túllépésnél 429-es hibát ad vissza.

5.3. API végpontok áttekintése

5. GET /health

A szolgáltatás állapotának ellenőrzése.

6. POST /generate-ideas

Több rövid posztötlet generálása iparág és célközönség alapján.

7. POST /generate-post

Strukturált közösségimédia-poszt generálása (hook, caption, cta, hashtags).

8. POST /generate-weekly-plan

7 napos tartalomterv készítése napi bontásban.

9. POST /generate-image

Marketingkép generálása stílus és méret paraméterekkel.

5.4. Bemeneti validáció

Az API minden generáló végpontnál Zod sémákat alkalmaz. A validáció ellenőrzi:

1. Kötelező mezők meglétét.
2. Típushelyességet.
3. Elfogadott értékkészletet (enum mezők).
4. Alapértelmezett értékek kitöltését opcionális mezőknél.

Érvénytelen kérés esetén a szerver 400 INVALID_INPUT hibát ad vissza, ezzel megelőzve a hibás AI-hívásokat és a bizonytalan kimeneteket.

5.5. Végpont-specifikus működés

/generate-ideas:

1. Bemenet validálása (industry, targetAudience, count, language).
2. AI mód esetén rövid ötletlista kérése.
3. Nem megfelelő AI-kimenet esetén fallback ötletek.

/generate-post:

1. Strukturált poszt készítése adott hangnemben.
2. JSON kimenet kényszerítése.
3. Hibás szerkezet esetén AI_BAD_JSON vagy AI_BAD_SHAPE válasz.

/generate-weekly-plan:

1. 7 napos terv generálása platform- és célspecifikus paraméterekkel.
2. Időtúllépés-védelem (Promise.race timeout).
3. Sikertelen első válasz esetén tömörített újrapróbálás.
4. Rövid/hibás kimenet esetén fallback heti terv.

/generate-image:

1. Promptépítés stílus, célcsoport és opcionális hely/korosztály alapján.
2. AI mód esetén képgenerálás.
3. Érvényes URL hiányában NO_IMAGE_URL vagy IMAGE_GEN_FAILED hiba.

5.6. Fallback stratégia és rendelkezésre állás

Az API egyik legfontosabb tervezési döntése a több szintű fallback működés. Ennek célja, hogy a felhasználó akkor is kapjon használható eredményt, ha:

1. az AI szolgáltatás nem elérhető,
2. a válasz formátuma hibás,
3. időtúllépés történik.

Ez a megközelítés javítja a rendszer megbízhatóságát, és jelentősen csökkenti a „üres állapot” típusú felhasználói hibákat.

5.7. Promptolási stratégia az API-ban

Az API a generálási feladatok során nem egyszerű szöveges kéréseket küld az AI-modellnek, hanem feladatspecifikusan felépített promptokat használ. Ezek a promptok tartalmazzák a felhasználó által megadott üzleti adatokat, valamint a kívánt válasz formájára és stílusára vonatkozó utasításokat is. Ennek köszönhetően a modell nemcsak tartalmilag releváns választ ad, hanem olyan szerkezetben, amelyet az alkalmazás közvetlenül fel tud dolgozni. Az ötlet- és posztgenerálásnál a rendszer JSON-alapú választ kér, míg a heti tartalomtervénél részletesebb instrukciók biztosítják a természetesebb és platformhoz igazított eredményt. A képgenerálás esetében a prompt a vizuális stílus, a célcsoport és az opcionális paraméterek alapján épül fel. A promptolás tehát az API működésének fontos része, mivel közvetlenül befolyásolja a generált tartalom minőségét és konzisztenciáját.

5.8. Környezeti konfiguráció

Az API működése környezeti változókon keresztül paraméterezhető:

1. PORT: szerverport.
2. USE_AI: AI mód be-/kikapcsolása.
3. OPENAI_API_KEY: OpenAI hozzáférés.

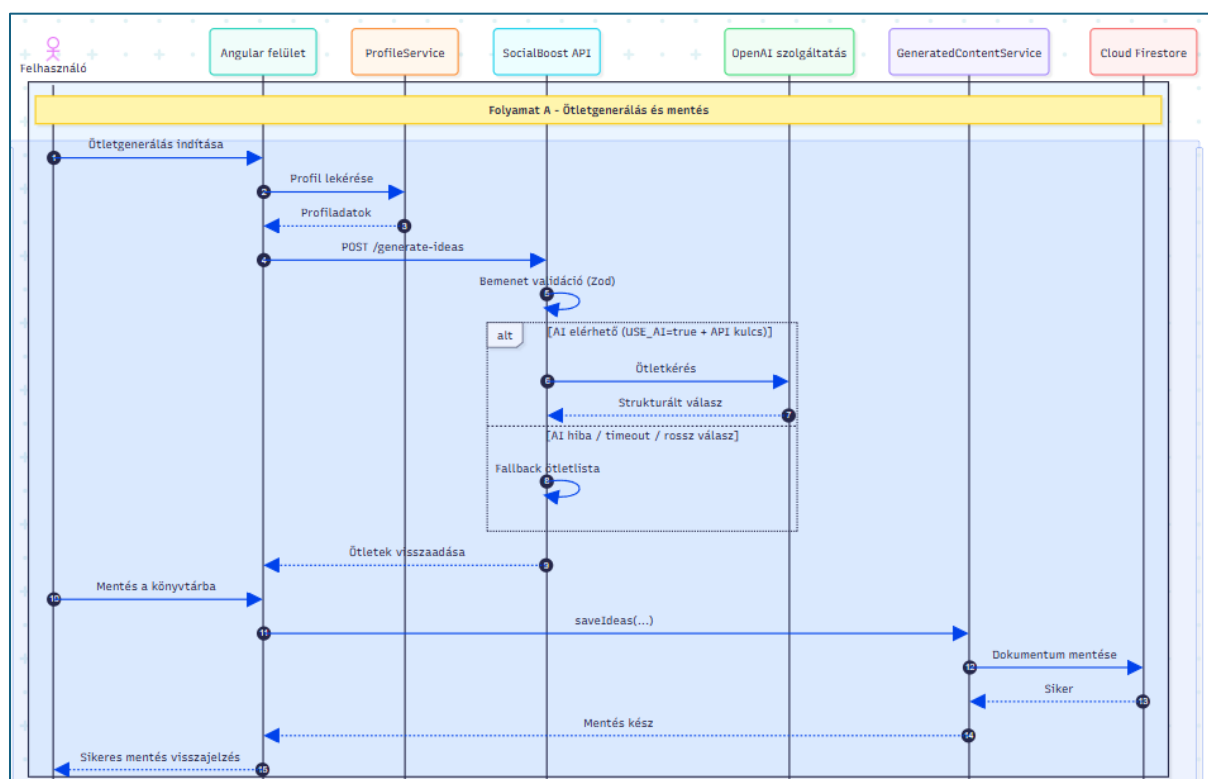
4. OPENAI_MODEL: használt nyelvi modell.

Ez lehetővé teszi, hogy fejlesztői, teszt és éles környezetben eltérő konfigurációval fusson azonos kódrendszer mellett.

5.9. Összegzés

A saját API nem csupán technikai összekötő réteg, hanem a rendszer stabilitásának és minőségének kulcseleme. A bemeneti validáció, a strukturált végpontok, a timeout- és fallback-mechanismusok, valamint a konfigurálható működés együtt biztosítják, hogy az alkalmazás valós felhasználói környezetben is megbízhatóan és kiszámíthatóan működjön.

6. A RENDSZER MAGASSZINTŰ FOLYAMATAI

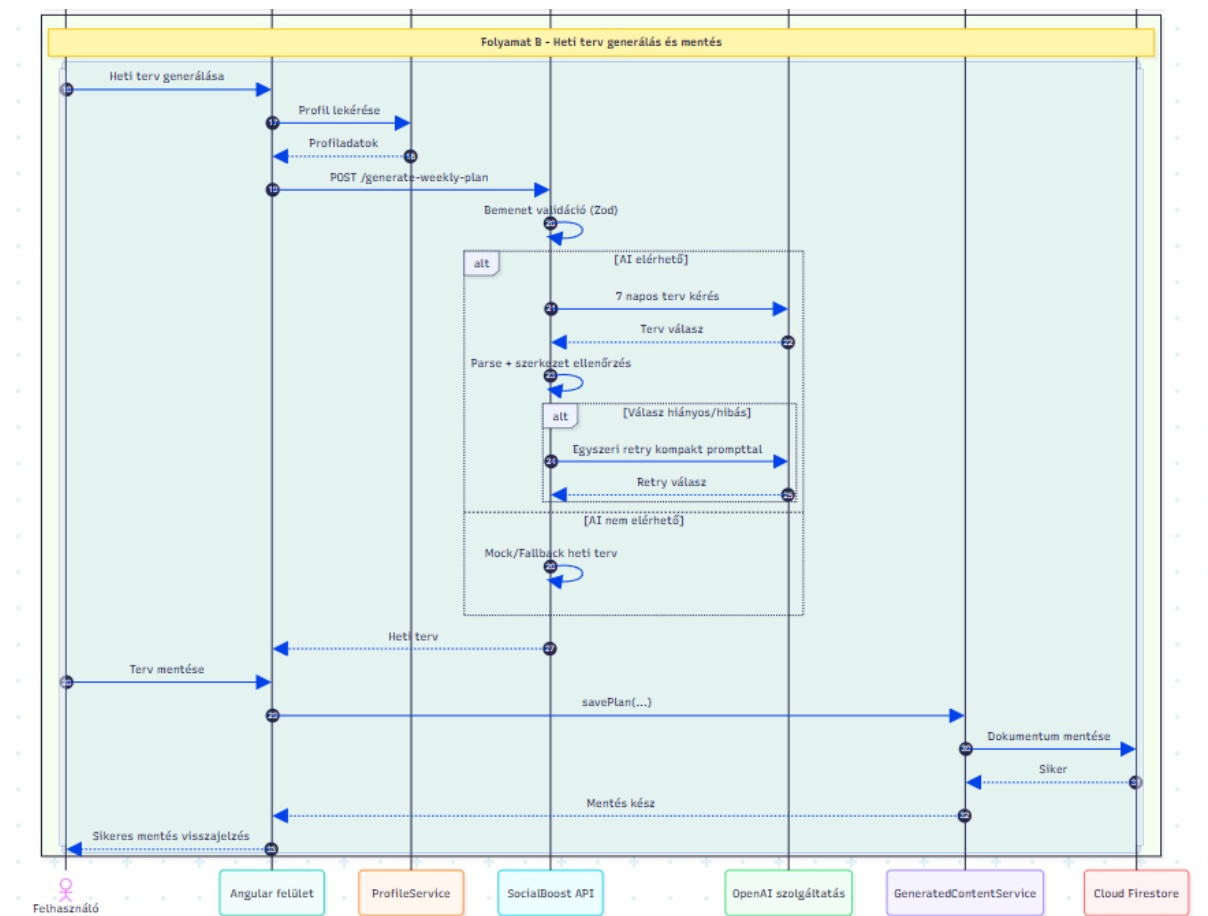


6.1 ábra: Ötletgenerálás magasszintű folyamata

A SocialBoost AI webalkalmazás magasszintű folyamatai közé tartozik az ötletgenerálás, a heti terv előállítás, valamint a generált tartalmak perzisztens mentése. A felhasználói működés központi eleme, hogy a rendszer minden generálási műveletet a felhasználó üzleti profiljának kontextusában hajt végre, majd az eredményt igény szerint Firestore adatbázisban eltárolja.

A 6.1-es és 6.2-es ábra két tipikus folyamatot mutat be: az ötletgenerálás és mentés, illetve a heti terv generálás és mentés szekvenciáját.

A két folyamat felépítése hasonló: a felhasználó kezdeményezi a műveletet, a kliensoldal lekéri a profiladatokat, majd API-hívást küld a backend felé. A backend minden esetben bemeneti validációt végez, és csak érvényes kérés esetén indítja el a generálást. AI elérhetőség esetén OpenAI hívás történik, hibás vagy hiányos válasz esetén pedig fallback mechanizmus lép életbe. Ennek köszönhetően a felhasználó szolgáltatáskieséskor sem marad eredmény nélkül.



6.2 Heti terv generálás magas szintű folyamata

A heti terv folyamat sajátossága, hogy a rendszer az első AI-válasz szerkezeti ellenőrzését követően szükség esetén egyszeri újrapróbálást végez tömörített prompttal, így javítva a 7 elemből álló terv teljességét és minőségét. A felhasználó ezután a kapott eredményt mentheti, amelyet a kliensoldali service réteg dokumentumként ír be a Firestore-ba. A mentési visszajelzés lezárja a folyamatot, és a tartalom később a könyvtár nézetben visszatölthető.

7. Mesterséges intelligencia használata a fejlesztés során

A SocialBoost AI rendszer fejlesztése és a szakdolgozat elkészítése során a mesterséges intelligencia több ponton is támogató eszközként jelent meg, azonban használata nem helyettesítette a tervezési, fejlesztési és ellenőrzési feladatokat, hanem azok hatékonyságát növelte. A fejlesztési folyamat során elsősorban GitHub Copilotot és ChatGPT-alapú asszisztenseket használtam. A GitHub Copilot főként a programozási környezetbe integrált fejlesztői támogatásként segítette a munkámat, különösen kisebb kódrészletek, ismétlődő minták, típusdefiníciók és egyszerűbb refaktorálási lépések elkészítésében. A ChatGPT-t ezzel szemben inkább magasabb szintű feladatokra alkalmaztam: architekturális ötletelésre, problémák szöveges átbeszélésére, hibák lehetséges okainak feltárására, valamint a szakdolgozati szöveg egyes részeinek nyelvi tömörítésére és átfogalmazására. A két eszköz tehát eltérő szerepet töltött be: a Copilot közvetlenül a kódírás közben volt hasznos, míg a ChatGPT inkább elemző és szövegező támogatóként működött.

A mesterséges intelligenciát több konkrét fejlesztési részfeladat során is felhasználtam. Az egyik ilyen terület a frontend és backend közötti interfészek kialakítása volt. Az Angular alapú kliensoldali service-ek, illetve a Node.js és Express alapú API-végpontok létrehozásánál az MI segített tipikus kódszerkezetek, request–response minták, típusdefiníciók és validációs sémák gyorsabb elkészítésében. Hasonló módon támogatta a Firebase Firestore-hoz kapcsolódó adattárolási logika, valamint a generált tartalmak kezelését végző service-ek kialakítását is. A Zod-alapú szerveroldali validációs sémák, a strukturált hibakezelés, illetve az OpenAI SDK hívási mintái esetén is hasznosnak bizonyult, hogy az MI gyorsan tudott javasolni kiinduló megoldásokat. Ezen felül az eszközöket refaktorálásnál is alkalmaztam, például amikor egy nagyobb komponenst vagy szolgáltatást kellett átláthatóbb, jobban tagolt formára bontani, illetve amikor a típuskezelést és a visszatérési szerkezeteket kellett egységesíteni.

A hibakeresés és a validáció területén az MI használata még hangsúlyosabb mérnöki kontrollt igényelt. A fejlesztés során az MI által adott javaslatokat nem kész megoldásként, hanem hipotézisként kezeltem. Ez azt jelentette, hogy minden érdemi kódjavaslatot futtatással, manuális ellenőrzéssel, valamint a projekt meglévő logikájával való összevetéssel validáltam. Több esetben előfordult, hogy az MI által javasolt megoldás első látásra helyesnek tűnt, de a konkrét rendszerkörnyezetben csak részben volt használható. Jó példa erre az, amikor a kliensoldali ötletgenerálási hívás több mezőt is továbbbított a backend felé, ugyanakkor a

szerveroldali séma ezek közül nem mindet dolgozta fel. Az ilyen eltérések felismeréséhez nem volt elegendő a generált kód elfogadása; szükség volt a frontend és backend tényleges összekapcsolásának végignézésére, valamint a requestek és validációs sémák összehasonlítására. Egy másik tanulságos terület a strukturált AI-válaszok kezelése volt. Az MI gyakran javasolt olyan megoldást, amely ideális esetben jól működik, de valós környezetben a válasz JSON-formátuma nem mindig stabil. Emiatt a rendszerben külön tisztító, kinyerő és fallback logikát kellett alkalmazni, amit nem lehetett kizárólag a generált kódra bízni. A validálás során ezért minden olyan pontot külön ellenőriztünk, ahol külső szolgáltatásból származó adat került feldolgozásra.

Az MI-outputok ellenőrzése több módszer kombinációjával történt. Egyrészt futtatási szinten vizsgáltam, hogy a javasolt megoldás valóban működik-e a projektben, például sikeres-e az API-hívás, helyes-e a válasz szerkezete, illetve megfelelően jelenik-e meg az adat az Angular komponensekben. Másrészt támaszkodtam a saját fejlesztői tudásomra is: különösen azokon a területeken, ahol a kód üzleti logikát, routingot, jogosultságkezelést vagy adatmodell-szintű döntéseket érintett. Emellett a Firebase, Angular és OpenAI dokumentációját is felhasználtam annak ellenőrzésére, hogy az adott javaslat megfelel-e a technológia valós működésének. Az MI tehát nem önmagában adott biztos választ, hanem gyorsabb kiindulópontot biztosított az elemzéshez és a megvalósításhoz.

Legalább ilyen fontos volt annak tudatosítása is, hogy a fejlesztés során mely részeket nem bíztam mesterséges intelligenciára. A rendszer alapvető koncepcionális döntéseit, például hogy a kliensoldal Angular standalone komponensekre épüljön, a backend külön REST API-ként működjön, illetve hogy a tartós adattárolás Firestore-ban történjen, nem az MI hozta meg helyettem. Ezeket a döntéseket a projekt követelményei, a technológiai ismereteim és a rendszer várható bővíthetősége alapján hoztam meg. Ugyanígy nem bíztam teljes egészében MI-re a szakdolgozat gondolati magját sem. A fejezetek szerkezetét, a rendszerbemutató hangúlyait, az architektúráis indoklásokat és az értékelő jellegű részeket saját megértés alapján fogalmaztam meg. Az MI itt legfeljebb stilisztikai segítséget adott, például tömörebb megfogalmazások vagy formálisabb mondatstruktúrák javaslatával. Szintén tudatos döntés volt, hogy az olyan részeknél, ahol személyes mérnöki értékelést kellett adni, például a rendszer erősségeiről, korlátairól vagy továbbfejlesztési lehetőségeiről, a végső szöveget saját megfogalmazásban készítettem el.

A fejlesztési folyamat során szerzett tapasztalatok alapján a mesterséges intelligencia használata egyértelműen gyorsította a rutinszerű és ismétlődő feladatokat. Gyorsabbá tette az

alapstruktúrák létrehozását, a típusok és interfészek megfogalmazását, a boilerplate jellegű kódrészek elkészítését, valamint a lehetséges hibák első körös azonosítását. Ugyanakkor több ponton lassította is a munkát, amikor a generált megoldás ugyan szintaktikailag helyes volt, de nem illeszkedett pontosan a projekt kontextusához. Ilyenkor az utólagos ellenőrzés, javítás és finomhangolás több időt vett igénybe, mint egy kisebb kódrészlet önálló megírása. A legfontosabb tanulság ezért az volt, hogy az MI-t nem szabad tekintélyalapú forrásként kezelni. A leghatékonyabb használati mód az volt, amikor kiindulópontként, gyors ötletforrásként vagy második nézőpontként vettem igénybe, de a végső döntést minden esetben saját elemzés és validáció alapján hoztam meg.

Összességében a mesterséges intelligencia használata a projekt során valódi fejlesztői produktivitásnövelő tényező volt, de csak abban az esetben, ha tudatos ellenőrzéssel és mérnöki kritikával párosult. A SocialBoost AI fejlesztésében az MI leginkább asszisztív eszközként bizonyult hasznosnak: segített gyorsabban eljutni a megoldás első változatáig, de a stabil, működő és szakmailag vállalható eredményhez minden esetben szükség volt manuális validációra, technológiai ismeretre és önálló döntéshozatalra. A folyamat egyik legfontosabb tanulsága az volt, hogy a mesterséges intelligencia nem kiváltja a fejlesztői gondolkodást, hanem akkor válik igazán értékké, ha azt kiegészíti és támogatja.

8. Tesztelés

Az alkalmazás főbb funkcióinak manuális tesztelése az alábbi táblázatban foglalható össze:

Funkció	Tesztelt lépések	Várt eredmény	Tényleges eredmény	Státusz
Regisztráció / bejelentkezés	Új fiók létrehozása, majd belépés	Sikeres hitelesítés, helyes átirányítás	Bejelentkezés sikeres, profil állapota szerint navigál	Megfelelt
Profil mentése	Onboarding mezők kitöltése, mentés, oldal újratöltése	Adatok Firestore-ban tárolva és visszatöltve	profile/main dokumentum létrejön, adatok visszaolvashatók	Megfelelt
Ötletgenerálás	Generálás indítása kitöltött profillal	Legalább 3 ötlet jelenik meg	Ötletlista megjelenik; AI nélkül fallback ötletek adódnak	Megfelelt
Strukturált poszt	Poszt generálás gomb megnyomása	Hook, caption, CTA és hashtagek megjelennek	Minden mező kitöltve jelenik meg, menthető	Megfelelt
Heti terv	Heti terv generálása, 45 mp timeout szimulálása	7 napos terv vagy fallback terv	Normál esetben 7 elem jelenik meg, timeoutnál fallback	Megfelelt

Funkció	Tesztelt lépések	Várt eredmény	Tényleges eredmény	Státusz
Képgenerálás	Stílus és méret kiválasztása, generálás	Kép URL visszaadása és megjelenítés	Kép URL érkezik és megjelenik; AI nélkül placeholder működik	Megfelelt
Kedvencek	Ötlet mentése, lista megtekintése, törlése	Kedvencek konzisztensen kezelhetők	Mentés, listázás és törlés Firestore-ban működik	Megfelelt
Védett oldal hozzáférés	Kijelentkezett állapotban védett URL megnyitása	Átírányítás a bejelentkezési oldalra	Auth guard aktiválódik, /login-ra navigál	Megfelelt
Érvénytelen API bemenet	Hiányos JSON küldése a generate-ideas végpontra	400-as INVALID_INPUT hibaválasz	Szerver 400-as választ ad, részletes hiba leírással	Megfelelt

8.1 Validáció röviden:

Bemeneti validáció az API oldalon Zod sémákkal történik; érvénytelen kérés esetén 400-as hiba adódik vissza.

Védett oldalak auth guarddal csak bejelentkezve érhetők el.

Fallback logika ellenőrzött USE_AI=false konfigurációval: minden generáló végpont mock adattal válaszol.

8.2 Automatizált tesztek és build eredmények:

Frontend build: `ng build --configuration production` — hibák nélkül lefutott, a disztribúciós csomag sikeresen előállt.

API build/typecheck: `npx tsc --noEmit` — TypeScript fordítási hibák nélkül átment, a szerver kódbázis típushelyessége megerősítve.

Frontend unit tesztek: `ng test` - 49/49 teszt sikeresen teljesített, 0 hiba.

Firestore security rules: `firebase emulators:exec --only firestore` — minden rules-teszt érvényes, jogosulatlan hozzáférés letiltva.

Irodalomjegyzék

1. „Angular” <https://angular.dev/> (Utolsó megtekintés: 2026.01.10.)
2. „TypeScript” <https://www.typescriptlang.org/docs/> (Utolsó megtekintés: 2026.01.10.)
3. „Draw.io” <https://app.diagrams.net/> (Utolsó megtekintés: 2026.04.15.)
4. „Firestore biztonsági szabályok” <https://firebase.google.com/docs/firestore/security/get-started> (Utolsó megtekintés: 2026.04.28.)
5. „Cloud Firestore Adatmodell” <https://firebase.google.com/docs/firestore/data-model> (Utolsó megtekintés: 2026.04.11.)
6. „Firebase Authentication dokumentáció” <https://firebase.google.com/docs/auth> (Utolsó megtekintés: 2026.04.14.)
7. „Express.js” <https://expressjs.com> (Utolsó megtekintés: 2026.03.03.)
8. „Node.js” <https://nodejs.org/en/about> (Utolsó megtekintés: 2026.05.13.)
9. „OpenAI API általános dokumentáció” <https://developers.openai.com/api/docs/concepts> (Utolsó megtekintés: 2026.05.13.)
10. „Stack Overflow” https://en.wikipedia.org/wiki/Stack_Overflow (Utolsó megtekintés: 2026.04.10.)
11. „GPT-4o-mini Modell leírás” <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/> (Utolsó megtekintés: 2026.04.10.)
12. „Openai API szövegenerálás útmutató” <https://developers.openai.com/api/docs/guides/text> (Utolsó megtekintés: 2026.04.11.)
13. „Zod” <https://zod.dev> (Utolsó megtekintés: 2026.04.15.)
14. „Predis.ai” <https://predis.ai/> (Utolsó megtekintés: 2026.04.15.)
15. „Canva” <https://www.canva.com> (Utolsó megtekintés: 2026.04.15.)
16. „Ocoya” <https://www.ocado.com> (Utolsó megtekintés: 2026.04.15.)
17. „Simplified” <https://simplified.com> (Utolsó megtekintés: 2026.05.04.)

Nyilatkozat

Alulírott Várnagy Erik Zoltán üzemmérnök-informatikus BProf szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztési Tanszékén készítettem, üzemmérnök-informatikus diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

2026. május 05.

Várnagy Erik Zoltán

Elektronikus melléklet

1. Az implementáció digitálisan megtekinthető a következő GitHub linken keresztül: <https://github.com/evarnagy/socialboost-ai> (Utolsó megtekintés: 2026.05.06.)
2. Hostolt alkalmazás linkje: <https://socialboost-ai-d44bc.web.app/> (Utolsó megtekintés: 2026.05.07.)
3. Végleges branch: main — Commit azonosító: [0f1df10](#)
4. API deploy URL (Render): <https://dashboard.render.com/project/prj-d7m8dvtckfvc73eb7k2g>